

select_ft_groups_by_silhouette.R

July 28, 2026

Contents

select_ft_groups_by_silhouette	1
Index	4

select_ft_groups_by_silhouette
<i>Select Optimal Number of Functional-Type Groups via Average Silhouette Width</i>

Description

Sweeps ‘number_of_ft_groups’ = ‘ft_groups_min’ .. ‘ft_groups_max’, cuts the dendrogram at each value, and returns the ‘number_of_ft_groups’ that maximises the average silhouette width computed from ‘dist_mat’.

The function also simulates the downstream pipeline filter chain for each candidate ‘k’ and only considers values that leave at least ‘min_n_taxa’ non-constant FT columns. The filter chain applied is: ‘filter_rare_taxa()’, ‘filter_community_by_n_cores()’, ‘filter_by_n_samples()’, ‘prepare_community_for_fit()’, optionally ‘compute_community_presence_absence()’ (when ‘error_family’ is “binomial”), and ‘filter_constant_taxa()’. The number of surviving columns is compared to ‘min_n_taxa’. Among viable candidates the highest-silhouette ‘k’ is returned. If no ‘k’ is viable an error is raised via ‘cli::cli_abort()’.

Usage

```
select_ft_groups_by_silhouette(  
  dist_mat,  
  hclust_obj,  
  ft_groups_min = 10L,  
  ft_groups_max = 25L,  
  data_community,  
  minimal_proportion,  
  min_n_taxa,  
  min_n_cores,  
  min_n_samples,  
  error_family  
)
```

Arguments

<code>dist_mat</code>	A 'dist' object produced by <code>'compute_dissimilarity_matrix()'</code> . Must inherit class <code>"dist"</code> .
<code>hclust_obj</code>	An 'hclust' object produced by <code>'fit_hclust()'</code> . Must inherit class <code>"hclust"</code> .
<code>ft_groups_min</code>	A single positive integer giving the minimum number of functional-type groups to evaluate. Must be at least 2 and no greater than <code>'ft_groups_max'</code> . Default: <code>'10L'</code> .
<code>ft_groups_max</code>	A single positive integer giving the maximum number of functional-type groups to evaluate. Must be at least 2. If <code>'ft_groups_max'</code> is greater than or equal to the number of observations, it is silently clamped to <code>'n_observations - 1L'</code> . After clamping <code>'ft_groups_max'</code> , <code>'ft_groups_min'</code> is silently clamped to <code>'min(ft_groups_min, ft_groups_max)'</code> to handle datasets with very few taxa without erroring. Default: <code>'25L'</code> .
<code>data_community</code>	A long-format data frame with columns <code>'taxon'</code> , <code>'dataset_name'</code> , <code>'age'</code> , and <code>'value'</code> , as produced by <code>'classify_to_functional_type()'</code> or <code>'classify_taxonomic_resolution()'</code> . Used to run the viability filter chain for each candidate <code>'k'</code> .
<code>minimal_proportion</code>	A single numeric value in (0, 1). A sample is considered to contain an FT group when the summed <code>'value'</code> for that group exceeds this threshold.
<code>min_n_taxa</code>	A single positive integer. The minimum number of non-constant FT columns (i.e. groups present in strictly between 0 <code>'minimal_proportion'</code>) that a candidate <code>'k'</code> must produce to be considered viable.
<code>min_n_cores</code>	A single positive integer. Forwarded to <code>'filter_community_by_n_cores()'</code> during the viability check to remove FT groups present in fewer than this many distinct cores (<code>'dataset_name'</code> values).
<code>min_n_samples</code>	A single positive integer. Forwarded to <code>'filter_by_n_samples()'</code> during the viability check to remove FT groups present in fewer than this many distinct spatio-temporal samples (<code>'(dataset_name, age)'</code> combinations).
<code>error_family</code>	A single character string (e.g. <code>"binomial"</code>). When <code>"binomial"</code> , <code>'compute_community_presence_absence'</code> is applied to the wide community matrix before <code>'filter_constant_taxa()'</code> during the viability check, mirroring the pipeline step.

Details

For each `'number_of_ft_groups'` in `'ft_groups_min'..ft_groups_max'` the function calls `'stats::cutree()'` followed by `'cluster::silhouette()'` and records the mean silhouette width.

For each candidate `'k'` the community data are aggregated to the FT level (summed `'value'` per `'(dataset_name, age, ft_group)'`) and then passed through the actual pipeline filter chain in sequence:

1. `'filter_rare_taxa()'` at `'minimal_proportion'`.
2. `'filter_community_by_n_cores()'`.
3. `'filter_by_n_samples()'`.
4. `'prepare_community_for_fit()'` (wide matrix, using all surviving samples as their own IDs).
5. `'compute_community_presence_absence()'` if `'error_family == "binomial"'`.
6. `'filter_constant_taxa()'`.

The number of surviving columns is the viability count. If any step errors (e.g. all taxa removed), the count is treated as zero.

Selection proceeds as follows:

1. If at least one 'k' has viability count $\geq$ 'min_n_taxa': return the highest-silhouette among those 'k' values.
2. If no 'k' is viable: abort with 'cli::cli_abort()'.

Ties are broken by 'base::which.max()' (first occurrence).

Value

A single integer giving the optimal number of functional-type groups ($\geq$ 'ft_groups_min' after clamping).

See Also

[compute_dissimilarity_matrix()], [fit_hclust()], [cluster_functional_types()]

Index

`select_ft_groups_by_silhouette, 1`